Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Bu ders beş öğrenci sunumundan oluşur. Sunumlar dersin mikroservis kitabının (Chris Richardson, *Microservices Patterns*) son bölümlerini izler: servisleri konteyner olarak dağıtmak (Service as a Container deseni ve Docker), monoliti mikroservislere yeniden yapılandırmak (nedenleri ve Strangler deseni), yeniden yapılandırma stratejileri, servis ile monolitin iş birliği (integration glue, Anti-corruption Layer, saga'larla veri tutarlılığı) ve önceki haftadan kalan bileşen (component) testleri. Sunan kişiler aşağıda "bir öğrenci" diye anılır. Hoca bu derste sunumları yönetti; sonunda mikroservis mimarisinin ana hatlarıyla, değişik yönleri ve desenleriyle incelenmesinin tamamlandığını söyledi.

Genel Konular

- Birinci sunum: Service as a Container deseniyle servis dağıtımı
  - Mikroservis mimarisinde çok sayıda servis vardır; her servisin farklı bağımlılıkları ve ortam ihtiyaçları olur. Bu, fiziksel sunucuya ya da sanal makineye kurulum yerine daha taşınabilir ve izole bir dağıtım yöntemi gerektirir. Bu desende servis, uygulamayı, bağımlılıklarını ve çalışma ortamını içeren bağımsız bir konteyner olarak paketlenir, ayağa kaldırılır ve dağıtılır.
  - Desenin katkıları: servislerin bağımsız dağıtılabilmesi, hızlı ölçekleme (yeni servis kaldırma ya da kapatma), farklı dil ve teknolojilerle geliştirilmiş servislerin işletim sistemi düzeyindeki farklı ihtiyaçlarının konteyner içinde kapsüllenmesi.
  - Dağıtım yöntemleri karşılaştırması (ekrandan, 00:06:45):
    - Geleneksel (sanal makine): her uygulama için bağımsız bir işletim sistemi çalışır; VM'lerin başlama süresi uzundur; kaynak tüketimi yüksektir. Katmanlar: Infrastructure → Hypervisor (VM Emulator) → her VM'de Guest Operating System, Bins/Libraries, Application.
    - Konteyner bazlı: tek bir işletim sistemi çekirdeği paylaşılır; hızlı başlar, kaynak tüketimi düşüktür; mikroservislerin küçük ve bağımsız yapısıyla uyumludur. Katmanlar: Infrastructure → Operating System → Container Engine → her konteynerde Bins/Libraries ve Application.
  - Konteynerlerin önemi (ekrandan, 00:10:00): geliştiriciler kodlarını hızlıca test edebilir ve tutarlı bir şekilde taşıyabilir; test ve üretim ortamları arasında tutarlılık sağlanır ("benim makinemde çalışıyordu" sorunu çözülür); DevOps süreçleri hızlanır, maliyet düşer. En yaygın konteyner teknolojileri: Docker, Solaris vb.
  - Docker: uygulamayı ve bütün bağımlılıklarını konteyner denen standart bir birime paketleyen açık kaynaklı platform. Sağladıkları: taşınabilirlik, tekrarlanabilirlik (test ve üretimde aynı davranış), verimlilik (fiziksel makinede çok sayıda işletim sistemi kurmaya gerek kalmaz), ölçeklenebilirlik.
  - Docker'ın ana bileşenleri (ekrandan, 00:12:20):
    - Docker Engine: konteynerlerin oluşturulması ve yönetimiyle ilgilenen çekirdek parça.
    - Docker Image: konteyner oluşturmak için kullanılan, uygulama kodunu ve bağımlılıklarını içeren salt okunur şablon; katmanlı (multi-layer) yapıdadır ve katmanlar imajlar arasında paylaşılır.
    - Docker Hub/Registry: konteyner imajlarını bulmak ve paylaşmak için bulut tabanlı depo (Java'daki Maven repository'ye, Node.js'teki npm'e benzer).
    - Dockerfile: Docker imajı oluşturmak için talimatlar içeren dosya.

    - Akış: Dockerfile –Build–> Image –Run–> Container; Image –Push–> Registry/Hub –Pull–> başka makinede Image –Run–> Container.
  - Dağıtım iki aşamalıdır: build (Dockerfile yazılır, servis ve bağımlılıkları imaja gömülür) ve runtime (imaj Docker Hub ya da AWS Elastic Container Registry gibi bir registry'ye konur, ihtiyaç duyan makine oradan çeker; Kubernetes gibi bir orkestrasyon aracı imajı istenen makinede ayağa kaldırır).
  - Dockerfile örneği, kitabın Listing 12.1'i, Restaurant Service (ekrandan, 00:16:20):

```
FROM openjdk:8u171-jre-alpine
RUN apk --no-cache add curl
CMD java ${JAVA_OPTS} -jar ftgo-restaurant-service.jar
HEALTHCHECK --start-period=30s --
    interval=5s CMD curl http://localhost:8080/actuator/health || exit 1
COPY build/libs/ftgo-restaurant-service.jar .
```

    - Temel imaj JRE'yi içeren küçük bir Linux (Alpine) imajıdır; curl sağlık kontrolü için kurulur; CMD konteyner başlayınca `java -jar` ile servisi çalıştırır; HEALTHCHECK `/actuator/health` uç noktasını ilk 30 saniyelik gecikmeden sonra 5 saniyede bir çağırır; COPY Gradle'ın build dizinindeki JAR'ı imaja kopyalar.
  - İmajı registry'ye gönderme (ekrandan, 00:17:45): `docker tag` versiyonlama ve isimlendirme (ör. `1.0.0.RELEASE`) için, `docker push` imajı registry adresine göndermek için kullanılır. Sonra başka bir makinede `docker run` ile imaj registry'den çekilip çalıştırılır.
  - Docker Compose: birden çok konteyneri tek bir YAML dosyasıyla tanımlayıp tek komutla başlatıp durdurmayı sağlar; geliştirmede kolaylık sağlar ama makineye özgü olduğu için üretimde yetersiz kalır. Üretimde farklı makinelerde servis çalıştırmak için orkestrasyon gerekir (en bilineni Kubernetes; Docker'ın aracı Docker Swarm).
  - Değerlendirme (ekrandan, 00:20:00): avantajlar izolasyon, kaynak kullanımı, hızlı dağıtım, taşınabilirlik, ölçeklenebilirlik, modern uygulama gereksinimlerine uyum; dezavantajlar sistem yönetimi yükü, orkestrasyon ihtiyacı, bağımlılıklar arası karmaşıklık, güvenlik riskleri (yanlış oluşturulmuş bir imaj bütün makinelere yayılır).
- İkinci sunum: mikroservislere yeniden yapılandırma (refactoring)
  - Sunum kitabın yeniden yapılandırma bölümünün girişini izler: mevcut monolitte mühendislik ekibi yeni özellikleri hızlı ve güvenilir şekilde sunamaz; çözüm olarak mikroservislere geçiş öne çıkar.
  - Neden bir monoliti yeniden yapılandıralım (ekrandan, 00:24:30): mikroservis mimarisinin bakımı, test edilebilirliği ve dağıtımı daha kolaydır, geliştirmeyi hızlandırır; daha ölçeklenebilirdir, hata izolasyonunu iyileştirir, teknoloji yığınını güncellemek kolaydır. Ancak monoliti yeniden yapılandırmak büyük bir iştir ve yeni özellik geliştirmeden kaynak çeker; bu yüzden önemli bir iş problemini çözmesi gerekir.
  - Monolitik cehennem: monolitik uygulamaların karşılaştığı sorunlar; yavaş teslimat (küçük değişiklik bile bütün uygulamanın yeniden dağıtımını gerektirir), hatalı yazılım sürümleri (test zorluğu), verimsiz ölçekleme (parçalar ayrı ayrı büyütülemez).
  - Önce sorunun gerçekten mimariden kaynaklanıp kaynaklanmadığı değerlendirilmelidir: yavaş teslimat ve hatalı sürümler çoğu zaman kötü geliştirme sürecinin sonucudur (ör. manuel testten otomatik teste geçmek hızı ciddi artırır); ölçeklenebilirlik sorunları bazen mimari değişmeden çözülebilir. Sorunlar sürerse mikroservis geçişi düşünülür.
  - Monoliti boğma (Strangler deseni): uygulama parça parça bağımsız mikroservislere dönüştürülür; yeni özellikler doğrudan mikroservis olarak geliştirilir, eski parçalar ihtiyaca göre tek tek taşınır. Sistem sürekli çalışır durumda kalır, kesintiler en aza iner.
  - Strangler uygulamasının stratejileri: hızlı değer katma (her geçiş adımı erken yatırım getirisi sağlar), yaygın değişikliklerden kaçınma (büyük değişiklik yerine küçük stratejik adımlar), altyapıya başta büyük yatırım yapmama (mikroservis deneyimi kazandıkça altyapıyı genişletme).
  - Çözüm her zaman mevcut problemlerin doğru analizine dayanmalı; bütün projeyi bir anda mikroservise geçirmek hata riskini artırır.
- Üçüncü sunum: monolitten mikroservislere geçiş stratejileri
  - Monolitik mimari: bütün bileşenlerin tek kod tabanında, sıkı bağlı olduğu, tek dağıtım birimli ve merkezi yönetimli yapı. Avantajları: basit geliştirme, performans (iletişim gecikmesi az), dağıtım ve bakım kolaylığı, yeni ve küçük projelere uygunluk. Dezavan-

tajları (uygulama büyüdükçe): karmaşıklık ve bağımlılık (değişikliklerin zincirleme etkisi), ekip iş birliğinin zorlaşması, gereksiz bütünsel ölçekleme, bakım zorluğu.
  - Strangler Fig modeli (ekrandan, 00:36:15): monolit tamamen yeniden yazılmadan dönüştürülür. Adımlar: yeni işlevler mikroservis olarak eklenir; eski bileşenler kademeli taşınır; zamanla bütün fonksiyonlar mikroservislere aktarılır. Bir süre monolit ve mikroservisler birlikte yaşar (coexistence), sonunda monolit kaldırılır. Avantaj: işlevsellik kesintiye uğramaz, dönüşüm aşamalıdır.
  - Yeni özellikleri servis olarak uygulama (ekrandan, 00:37:30): monolitin büyümesini engeller, yeni özelliklerin hızlı geliştirilip dağıtılmasını sağlar, mikroservislerin değerini hızla gösterir. Entegrasyon için iki araç: API Gateway (eski özellik isteklerini monolite, yeni özellik isteklerini yeni servise yönlendirir) ve integration glue kodu (servisin monolitin verisine erişmesini ve monolitteki işlevselliği çağırmasını sağlar). Slayttaki örnekte yeni servis DelayedDeliveryService, Order ve Notification aggregate'lerini içerir.
  - Sunum katmanını arka uçtan ayırma (ekrandan, 00:38:45): katmanlar sunum (kullanıcı arayüzü), iş mantığı (iş kuralları) ve veri erişimi (veritabanı ve mesajlaşma). Sunum mantığı ile iş mantığı arasındaki kaba taneli API doğal bir dikiş yeridir; monolit, bağımsız dağıtılabilen bir sunum uygulaması (web app + REST client) ve bir arka uç monolitine (REST API + iş mantığı + veritabanı) bölünür. Faydaları: bağımsız geliştirme ve dağıtım, A/B testi ve kullanıcı deneyiminde hızlı iterasyon.
  - İşlevsel yeteneklerin servislere taşınması (ekrandan, 00:40:00): monolit küçülür, iş mantığı daha iyi ölçeklenir; API uç noktaları, iş mantığı ve veritabanı erişim mantığı dikey dilimler hâlinde bağımsız taşınır. Örnekte Order Service'in Courier ve Plan aggregate'lerini içeren kısmı yeni bir servise çıkarılır; monolit ile servis integration glue (inbound/outbound adapter'lar) ile bağlanır.
  - Zorluk 1, domain modelini parçalama (ekrandan, 00:41:20): servis sınırlarını aşan nesne referansları sorun çıkarır. Örnekte Delivery Service'e taşınan Order entity'si monolitteki Restaurant entity'sine referans verir. Çözüm: DDD aggregate'lerinde olduğu gibi nesne referansı yerine birincil anahtar kullanmak; Order'daki Restaurant referansı `restaurantId` alanıyla değiştirilir.
  - Zorluk 2, veritabanını yeniden yapılandırma (ekrandan, 00:44:25): domain sınıfları kalıcı olduğu için servis çıkarılırken tablolar da taşınır ya da bölünür. Örnekte ORDER tablosundaki teslimatla ilgili sütunlar (scheduledPickupTime, scheduledDeliveryTime) yeni DELIVERY tablosuna taşınır; Delivery Service bu verileri monolite replike eder ve monolitteki Order'da bu alanlar salt okunur kalır. Böylece monolitte hemen yapılması gereken değişiklik azalır. Geçişte replikasyon ve salt okunur proxy'ler veri tutarlılığını korur.
  - Bağımlılık yönetimi: her mikroservis bağımsız hâle getirilir, iletişim açıkça tanımlı API'ler üzerinden yapılır.
  - Geçişin faydaları: hızlı geliştirme ve dağıtım, bileşen bazında ölçekleme (maliyet düşer), hata izolasyonuyla artan güvenilirlik, teknoloji çeşitliliği. Zorluğu: teknik ve organizasyonel değişiklik gereksinimi.
- Dördüncü sunum: servis ile monolitin iş birliği tasarımı
  - Sunum, iki yazılım mimarisi kanunuyla açılır: yazılım mimarisindeki her şey bir ödünleşimdir (trade-off); neden, nasıldan daha önemlidir.
  - Monolitten servis ayırma, yeni özellik eklenirken ya da monolitin performansı düştüğünde sorumluluklarını mikroservislere aktarmak için yapılır. Ayrılan iki bileşenin haberleşmesi gerekir; kitap bu iletişim koduna integration glue der. Slayttaki kitap şekline göre glue, API'leri uygulayan adapter'lardan oluşur; bazı API'ler mesajlaşma, bazıları RPC tabanlıdır (ekrandan, 00:50:00).
  - Integration glue tasarım adımları (ekrandan, 00:50:00): (1) mikroservisin hangi API'leri sunacağına karar verilir; (2) API kullanıcılarının prosedürün iç mantığını bilmesine gerek yoktur, iç mantık gizlenir (encapsulation); (3) entegrasyon stili ve IPC mekanizması seçilir.
  - Kitaptaki örnek (ekrandan, 00:51:20): Delayed Delivery Service, CustomerContactInfoRepository arayüzünü uygulayan bir REST client ile monolitin REST API'sini çağırır (`GET /customers/{customerId}`). REST geliştirme açısından basittir ama performansı yüksek değildir; monoliti meşgul edebilir; senkron IPC iki bileşeni de yavaşlatabilir. Sorgu çok sıksa alternatif tasarım: monolit event'leri bir mesaj kanalına yazar, servis CQRS ile kendi veritabanında bir kopya tutar. Mesaj kanalı sistemi karmaşıklaştırır

ama monolitin yükünü azaltır.
  - Saga'larda izolasyon eksikliği: SQL veritabanları ACID sağlar, saga'lar yalnız ACD sağlar (izolasyon yoktur).
  - Eski monolitin eski diller ve kütüphanelerle yazılmış olması, ekipte bilen kalmaması da dezavantajdır. Monolitin domain modelini değiştirmek risklidir; kitabın önerdiği yol iki modeli birlikte kullanmaktır.
  - Anti-corruption Layer deseni (ekrandan, 00:53:50): servis yeni domain modelini, monolit eski modelini kullanır; aradaki çeviri ACL ile yapılır. Şekilde Delayed Delivery Service'in API'si ile monolitin REST API'si (`GET /user/{userId}`) arasında Translation layer, Monolith layer ve REST client katmanları vardır. ACL servis tarafına konur (monolite dokunmamak için). Eski domain modelinin yeni modeli kirletmesini önler; OOP'deki Adapter desenine benzer. ACL farklı bileşenlerde olabilir, iletişim yönü de farklı olabilir.
  - Saga terimleri (ekrandan, 00:55:10): saga (asenkron mesajlaşmayla koordine edilen yerel işlemler dizisi), compensating transaction (yerel işlemin güncellemelerini geri alan işlem), countermeasure (saga'lar arası izolasyon eksikliğini ele alan tasarım tekniği), semantic lock (saga'nın güncellediği kayda bayrak koyan countermeasure), compensatable transaction (sonrasındaki bir işlem başarısız olabileceği için telafi gerektiren işlem), pivot transaction (saga'nın dönüm noktası; başarılı olursa saga sonuna kadar çalışır), retriable transaction (pivottan sonra gelen ve başarısı garanti olan işlem).
  - Kitchen Service örneği (ekrandan, 00:56:20): Kitchen Service monolitten çıkarılırken Order entity'si bölünür ve Kitchen Service'te Ticket entity'si oluşturulur; bu, monolitin `createOrder()` gibi birçok komutunu etkiler. Monolitte tek ACID işlemi olan `createOrder()` (sipariş ayrıntılarını doğrula, tüketicinin sipariş verebildiğini doğrula, kredi kartını yetkilendir, Order oluştur) bir saga ile değiştirilir: (1) monolitte Order APPROVAL_PENDING durumunda oluşturulur ve tüketici doğrulanır; (2) Kitchen Service'te sipariş doğrulanır, Ticket CREATE_PENDING durumunda oluşturulur; (3) monolitte kredi kartı yetkilendirilir, Order APPROVED olur; (4) Kitchen Service'te Ticket AWAITING_ACCEPTANCE durumuna geçer. APPROVAL_PENDING ve CREATE_PENDING durumları semantic lock örnekleridir. Öğrenciye göre servislerin çıkarılma sırası, monolitte compensating transaction yazma ihtiyacını azaltabilir; sonraki adım başarısız olmuyorsa telafi işlemine gerek kalmaz.
  - Kimlik doğrulama ve yetkilendirme: monolit ile mikroservisin farklı yöntemleri API Gateway'de toplanır; gateway monolite dokunmadan ikisini birlikte yönetir.
- Beşinci sunum: bileşen (component) testleri (önceki haftadan kalan sunum)
  - Bileşen testi, bir bileşeni izole olarak, dış API'si üzerinden test eder. Test piramidinde entegrasyon testleri ile end-to-end testler arasında yer alır. Bileşenin bağımlılıkları stub ya da mock nesnelerle simüle edilir.
  - Stub (ekrandan, 01:00:00): test edilen kodun bağımlı olduğu bileşenin yerine kullanılan, sabit (önceden tanımlı) davranış sergileyen test çifti. Belirli girdilere sabit cevap döner, dış bağımlılıkları izole eder, çağrıları kaydetmez ya da analiz etmez. Örnek: bir API'den veri çeken fonksiyonu gerçek API olmadan test etmek.
  - Mock: bileşeni simüle ederken kendisine yapılan çağrıları kaydeden ve analiz eden test çifti; dış bileşenle etkileşimin doğruluğunu kontrol eder. Örnek: e-posta gönderim fonksiyonunun doğru parametrelerle çağrıldığını test etmek.
  - Kabul testleri: yazılımın gereksinimleri ve kullanıcı ihtiyaçlarını karşıladığını doğrulayan son aşama testleri; senaryolar kullanıcıya göre belirlenir.
  - Gherkin: BDD'de test senaryolarını teknik olan ve olmayan ekiplerin anlayacağı biçimde yazmak için kullanılan DSL; Given (başlangıç durumu), When (eylem), Then (beklenen sonuç). Cucumber, Gherkin'i koda bağlar.
  - Gherkin örneği (ekrandan, 01:03:40): "Place Order" özelliğinde iki senaryo; geçerli kartla sipariş APPROVED olur ve OrderAuthorized event'i yayınlanır; süresi dolmuş kartla sipariş REJECTED olur ve OrderRejected event'i yayınlanır. StepDefinitions sınıfında `@Given`, `@When`, `@Then`, `@And` açıklamalı yöntemler adımları koda bağlar.
  - Bileşen testi tasarımı: Gherkin senaryosunu çalıştırmadan önce Order Service başlatılmalı ve bağımlılıkları (veritabanı, mesajlaşma altyapısı) ayarlanmalıdır.
    * In-process bileşen testi: servis, bağımlılıkları bellek içi stub ve mock'larla değiştirilerek aynı JVM içinde çalışır (HSQLDB gibi). Yazması daha basit ve hızlıdır, ama dağıtılabilir servisi test etmez.

    * Out-of-process bileşen testi: servis üretime hazır biçimiyle (genelde Docker konteyneri) ayrı süreçte çalışır; gerçek altyapı servisleri (Order Service için MySQL ve Apache Kafka) kullanılır, uygulama servisi bağımlılıkları (Consumer Service, Accounting Service) stub'lanır. Daha gerçekçi ve kapsamı yüksektir, ama daha yavaş ve kırılgandır.
  - FTGO Order Service bileşen testleri (ekrandan, 01:07:15): `OrderServiceComponentTest` sınıfı `@RunWith(Cucumber.class)` ve `@CucumberOptions(features = "src/component-test/resources/features")` açıklamalarına sahiptir; test yöntemi içermez, Gherkin özelliklerini okuyup `OrderServiceComponentTestStepDefinitions` sınıfıyla yürütür. Bu sınıf Spring Testing'in `@ContextConfiguration` açıklamasını taşır.
  - Step definitions (ekrandan, 01:09:20): `useCreditCard()` yöntemi `SagaParticipantStubManager` ile Accounting Service stub'ını kart "valid" ise başarı, "invalid" ise başarısızlık cevabı verecek şekilde yapılandırır; `placeOrder()` Order Service'in REST API'sini çağırıp cevabı sonraki adımda doğrulamak için saklar; `baseUrl()` servisin URL'sini döndürür.
  - Testlerin çalıştırılması (ekrandan, 01:10:00): Docker Compose ile Order Service ve altyapısı başlatılır; Gradle Docker Compose eklentisi testlerden önce konteynerleri başlatır, sonra durdurur. Testler yavaş olduğu için `./gradlew test` içinde değil, `src/component-test/java` dizininde tutulup `./gradlew componentTest` ile çalıştırılır; `componentTest` görevi JAR önce derlensin diye `assembly`'ye bağımlıdır. Adımlar: Order Service'i derle, servisi ve altyapısını başlat, testleri çalıştır, servisleri durdur.

Hocanın Özellikle Vurguladığı Kısımlar

- Bu derste hoca akademik bir vurgu yapmadı; sunumları sırayla yönetti. Dersin sonunda mikroservis mimarisinin ana hatlarıyla, değişik yönleri ve desenleriyle incelenmesinin tamamlandığını söyledi. Sunumlarda öne çıkan noktalar aşağıdadır.
- Konteyner, sanal makineye göre hafiftir
  - Konteynerler tek işletim sistemi çekirdeğini paylaşır; VM'ler her biri kendi işletim sistemini çalıştırır.
- Monolitten mikroservise geçiş aşamalı olmalıdır
  - Strangler deseni: yeni özellik servis olarak yazılır, eski işlevler kademeli taşınır, sistem sürekli çalışır.
- Geçişten önce sorunun kaynağı analiz edilmelidir
  - Yavaş teslimat ve hatalı sürümler çoğu zaman mimari değil süreç sorunudur.
- Servis sınırını aşan nesne referansları birincil anahtarla değiştirilir.
- Anti-corruption Layer eski domain modelinin yeni modeli kirletmesini önler.
- Stub ile mock farkı
  - Stub sabit veri sağlar, çağrıları kaydetmez; mock çağrıları kaydeder ve etkileşimi doğrular.

Kısa Tekrar Notları

- Service as a Container: servis + bağımlılıklar + çalışma ortamı tek konteynerde; bağımsız dağıtım, hızlı ölçekleme.
- VM: Infrastructure → Hypervisor → Guest OS (her VM'de); konteyner: Infrastructure → OS → Container Engine.
- Docker bileşenleri: Engine, Image (salt okunur, katmanlı şablon), Hub/Registry, Dockerfile.
- Akış: Dockerfile → build → image → push → registry → pull → run → container.
- `docker tag` versiyonlar, `docker push` registry'ye gönderir, `docker run` çeker ve çalıştırır.
- Docker Compose geliştirme içindir; üretimde orkestrasyon (Kubernetes, Docker Swarm).
- Konteyner dezavantajları: yönetim yükü, orkestrasyon ihtiyacı, bağımlılık karmaşıklığı, güvenlik riski.
- Monolitik cehennem: yavaş teslimat, hatalı sürümler, verimsiz ölçekleme.
- Strangler (Fig): yeni işlev servis olarak; eski işlev kademeli taşınır; sonunda monolit kaldırılır.
- Strangler stratejileri: hızlı değer katma, yaygın değişiklikten kaçınma, altyapıya başta

büyük yatırım yapmama.
- Üç strateji: yeni özellikleri servis olarak uygulama, sunum katmanını arka uçtan ayırma, işlevsel yetenekleri servise çıkarma.
- API Gateway eski/yeni istekleri yönlendirir; integration glue servis ile monoliti bağlar.
- Nesne referansı → birincil anahtar (`restaurantId`).
- Tablo bölme + replikasyon + salt okunur alanlar.
- Integration glue tasarımı: API seçimi, encapsulation, entegrasyon stili ve IPC seçimi.
- REST basit ama senkron ve monoliti yorar; alternatif: mesaj kanalı + CQRS kopya.
- Saga'lar ACD sağlar, izolasyon sağlamaz → countermeasure'lar (semantic lock vb.).
- Pivot transaction: go/no-go noktası; sonrasındaki retriable işlemler başarılı olur.
- ACL: Translation layer + Monolith layer + REST client; servis tarafında; Adapter desenine benzer.
- Kimlik doğrulama/yetkilendirme farkları API Gateway'de toplanır.
- Bileşen testi: izole, dış API üzerinden; bağımlılıklar stub/mock.
- Stub sabit cevap, çağrı kaydetmez; mock çağrı kaydeder.
- Gherkin: Given/When/Then; Cucumber step definitions ile koda bağlanır.
- In-process: aynı JVM, hızlı, basit, dağıtılabilir servisi test etmez; out-of-process: Docker, gerçek altyapı, gerçekçi ama yavaş.
- `./gradlew componentTest`, `src/component-test/java`, Gradle Docker Compose eklentisi.

Detaylı Açıklamalar

- Neden konteyner?
  - Mikroservislerde onlarca servis farklı dil ve kütüphanelerle yazılabilir; bunları aynı makinede çalıştırmak işletim sistemi düzeyinde çakışmalara yol açabilir. Sanal makine her servise ayrı işletim sistemi verir ama ağırdır: bir işletim sistemi 20-30 GB yer kaplayabilir ve başlaması uzun sürer. Konteyner ise host'un çekirdeğini paylaşır, ortak imaj katmanlarını yeniden kullanır; bu yüzden tek fiziksel sunucuda çok sayıda konteyner hızla kaldırılabilir. Servisin ihtiyacı olan her şey konteynerin içinde olduğu için test ve üretim aynı davranır.
- Build ve runtime
  - Build aşamasında Dockerfile'daki komutlarla imaj üretilir; örnekte JRE içeren küçük bir temel imaja curl kurulur, servis JAR'ı kopyalanır, başlangıç komutu ve sağlık kontrolü tanımlanır. Runtime aşamasında imaj etiketlenip bir registry'ye gönderilir; servisi çalıştıracak makine imajı oradan çeker. Tek makinede birden çok konteyner Docker Compose ile yönetilebilir; birden çok makinede ise Kubernetes gibi bir orkestratör gerekir.
- Geçiş kararı ve Strangler
  - Monoliti mikroservise çevirmek pahalıdır, bu yüzden gerçek bir iş problemiyle gerekçelendirilmelidir. Önce sorunların süreçten mi mimariden mi kaynaklandığına bakılır. Geçiş kararı verilirse "büyük patlama" yerine Strangler deseni uygulanır: monolitin çevresinde yeni servisler büyür, monolit küçülür ve sonunda kaybolur. Her adım kendi başına değer üretir ve risk dağılır.
- Üç geçiş stratejisi
  - Yeni özellikler servis olarak yazılır; API Gateway yeni istekleri servise, eskileri monolite yönlendirir. Sunum katmanı ayrılarak monolit ikiye bölünür; iki parça bağımsız dağıtılır. İşlevsel yetenekler (API uç noktası, iş mantığı ve veri erişimi birlikte, dikey dilim olarak) servislere çıkarılır. Her durumda servis ile monolit arasındaki bağlantıyı integration glue kurar.
- Servis çıkarmanın zorlukları
  - Domain modelinde sınıflar birbirine nesne referanslarıyla bağlıdır; bir sınıf servise taşınınca referans servis sınırını aşar. Çözüm referansı birincil anahtarla değiştirmektir. Veritabanında da tablolar bölünür; yeni servisin verisi geçiş sürecinde monolite replike edilir, böylece monolitteki kod hemen değiştirilmek zorunda kalmaz.
- Monolit ile servisin iş birliği
  - Servis monolitin verisine REST ile senkron erişebilir; bu basittir ama monoliti yorar ve iki tarafı da yavaşlatabilir. Sorgu sıksa monolit event yayınlar, servis CQRS ile kendi kopyasını tutar. İki domain modeli arasında çeviri ACL ile yapılır ve ACL servis tarafına konur. Veri tutarlılığı saga'larla sağlanır; saga'larda izolasyon olmadığı için semantic lock gibi countermeasure'lar kullanılır. Kitchen Service örneğinde mono-

litteki tek ACID işlemi, monolit ile servis arasında gidip gelen bir saga'ya dönüşür; servislerin çıkarılma sırası monolitte telafi işlemi yazma ihtiyacını etkiler.

- Bileşen testleri
  - Bileşen testi tek bir servisi, dış API'si üzerinden ama izole olarak test eder. Bağımlılıklar stub ya da mock'la simüle edilir. Senaryolar Gherkin ile yazılır ve Cucumber step definitions ile koda bağlanır. Out-of-process testte servis Docker konteynerinde, gerçek MySQL ve Kafka ile çalışır; diğer servisler SagaParticipantStubManager ile stub'lanır. Bu testler yavaş olduğu için ayrı bir Gradle görevi olarak çalıştırılır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.